

TypeScript + Zod + Arquitectura Limpia.

Proyecto: ArchiCode - Sistema de Gestión de Arquitectura de Software.

Introducción.

Hasta ahora venimos construyendo aplicaciones/sistemas que funcionan, pero ¿cómo nos aseguramos de que sigan funcionando bien cuando crezcan? ¿Cómo evitamos que un cambio acá rompa algo allá? ¿Cómo validamos que los datos que nos llegan son correctos sin escribir 100 `if`s?

Hoy vamos a dar un salto de calidad profesional. Vamos a combinar tres herramientas poderosas:

- TypeScript → Para que el editor nos ayude a evitar errores de tipos ANTES de ejecutar el código.
- Zod → Para validar datos en tiempo real (ej: lo que llega de un formulario o API).
- Arquitectura Limpia / Hexagonal → Para organizar el código en capas, separando la lógica de negocio de los detalles técnicos (base de datos, interfaces de usuario, APIs externas).

Al final de la clase, van a poder diseñar, documentar y validar arquitecturas de software con tipado fuerte. Eso es nivel ingeniería.

TypeScript más allá de lo básico.

Ya saben tipos de datos básicos (`string`, `number`, `boolean`). Ahora vamos a lo potente:

// Interfaces: contrato de lo que debe tener un objeto

```
interface Arquitecto {  
  id: number;  
  nombre: string;  
  especialidad: 'frontend' | 'backend' | 'devops';  
}
```

// Utility Types: transforman tipos existentes

```
type ArquitectoSinId = Omit<Arquitecto, 'id'>; // saca 'id'  
type ArquitectoParcial = Partial<Arquitecto>; // todos opcionales
```

```
// Generics: reutilizo lógica para distintos tipos
function guardarEntidad<T>(entidad: T): T {
  console.log(`Guardando: ${JSON.stringify(entidad)}`);
  return entidad;
}
```

Zod - Validación en runtime.

TypeScript ayuda en tiempo de escritura, pero cuando llegan datos de afuera (formulario, API, localStorage) necesitamos validar en vivo.

```
import { z } from 'zod';

// Defino el esquema
const ArquitectoSchema = z.object({
  nombre: z.string().min(3, "Mínimo 3 caracteres"),
  edad: z.number().int().positive().max(120),
  email: z.string().email(),
  especialidad: z.enum(['frontend', 'backend', 'devops'])
});

// Uso: valido datos inseguros
const datosInseguros = { nombre: "Ana", edad: 25, email: "ana@mail.com", especialidad: "backend" };
const resultado = ArquitectoSchema.safeParse(datosInseguros);

if (resultado.success) {
  console.log("Datos válidos", resultado.data);
} else {
  console.log("Errores:", resultado.error.errors);
}
```

Arquitectura Hexagonal (Puertos y Adaptadores).

Imaginemos un enchufe europeo vs argentino. El puerto es la interfaz (ej: "tomacorriente de 220V"), el adaptador es el convertidor físico. En código:

- **Puerto (interfaz):** define QUÉ quiero hacer (ej: `guardarArquitecto`)
- **Adaptador (implementación):** define CÓMO se hace (ej: `guardarEnLocalStorage`, `guardarEnMongoDB`)

Esto permite cambiar de base de datos sin tocar la lógica de negocio.

Estructura de carpetas:

src/

```
├─ core/          # Núcleo: no depende de nada externo
|  ├─ entidades/   # interfaces y tipos puros
|  ├─ puertos/     # abstracciones (repositorios, servicios)
|  └─ casos-uso/   # lógica de negocio
└─ infraestructura/ # detalles concretos
    ├─ adaptadores/ # implementaciones reales (localStorage, API)
    └─ react/       # componentes UI
```

Ejemplo práctico.

Para ver como se aplica se demuestra el ejemplo de un Sistema de Validación de Recetas Médicas.

MedCheck - Validador de Recetas.

Requisitos del ejemplo:

- Validar que una receta tenga: paciente (string, no vacío), medicamento (string), dosis (número positivo)
- Guardar la receta (simulado) y mostrar errores bonitos.

// 1. Entidad pura (core/entidades/receta.ts)

```
export interface Receta {
  paciente: string;
  medicamento: string;
  dosis: number;
}
```

```
// 2. Esquema Zod (infraestructura/validacion/recetaSchema.ts)
import { z } from 'zod';

export const RecetaSchema = z.object({
  paciente: z.string().min(1, "Nombre del paciente requerido"),
  medicamento: z.string().min(1),
  dosis: z.number().positive("La dosis debe ser mayor a 0")
});
```

```
// 3. Puerto (core/puertos/repositorioRecetas.ts)
export interface RepositorioRecetas {
  guardar(receta: Receta): Promise<void>;
}
```

```
// 4. Adaptador concreto (infraestructura/adaptadores/localStorageRepo.ts)
import { RepositorioRecetas } from '../core/puertos/repositorioRecetas';

export class LocalStorageRepo implements RepositorioRecetas {
  async guardar(receta: Receta): Promise<void> {
    const existentes = JSON.parse(localStorage.getItem('recetas') || '[]');
    existentes.push(receta);
    localStorage.setItem('recetas', JSON.stringify(existentes));
  }
}
```

```
// 5. Caso de uso (core/casos-uso/regarReceta.ts)
import { Receta, RepositorioRecetas } from '...';
import { RecetaSchema } from '...';

export async function registrarReceta(
  datosDesconocidos: unknown,
  repositorio: RepositorioRecetas
): Promise<{ exito: boolean; error?: string }> {
  const validacion = RecetaSchema.safeParse(datosDesconocidos);
  if (!validacion.success) {
    return { exito: false, error: validacion.error.errors[0].message };
  }
  await repositorio.guardar(validacion.data);
}
```

```
return { exito: true };  
}
```

¿Ven? La lógica de negocio (`registrarReceta`) NO sabe si guardamos en localStorage, MongoDB o una API. Solo conoce el puerto (`RepositorioRecetas`). Eso es Arquitectura Limpia.

Proyecto principal de la clase: ArchiCode.

Descripción del proyecto: ArchiCode es una plataforma web donde arquitectos de software pueden:

- Diseñar componentes de software (con nombre, tipo, dependencias)
- Validar que sigan patrones conocidos (Hexagonal, MVC, etc.)
- Obtener feedback en tiempo real sobre su diseño

Temática: Van a construir un "validador de arquitectura" como los que usan equipos de Netflix o Spotify para mantener orden en sus microservicios.

Funcionalidades obligatorias.

1. Formulario para crear un nuevo componente arquitectónico (con tipado fuerte TS + Zod).
2. Listado de componentes agregados.
3. Validación en tiempo real: si un componente viola una regla (ej: dependencia circular), mostrar error.
4. Persistencia en localStorage (usando adaptador hexagonal).
5. Feedback visual moderno (Tailwind opcional, pero pueden usar CSS puro).

Estructura esperada del código (mínima).

```
archicode/  
├── index.html  
├── src/  
│   ├── core/  
│   │   ├── entidades/  
│   │   │   ├── ComponenteArquitectura.ts # interface y tipo  
│   │   │   ├── puertos/  
│   │   │   ├── repositorioComponentes.ts # interfaz  
│   │   ├── casos-uso/  
│   │   └── agregarComponente.ts
```

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

```
| | └─ validarPatron.ts
| | └─ infraestructura/
| | └─ adaptadores/
| | └─ localStorageRepo.ts
| | └─ validacion/
| | └─ componenteSchema.ts
| | └─ react/ (o vanilla JS con manipulación DOM)
| | └─ ui.js
| | └─ main.js
| └─ main.js (punto de entrada)
└─ package.json (con TypeScript, Zod, Vite opcional)
└─ tsconfig.json
```

Requisitos técnicos.

- TypeScript estricto (`strict: true` en tsconfig).
- Al menos 3 utility types distintos (ej: `Partial`, `Pick`, `Omit`, `Readonly`).
- Al menos 1 generic en una función o interfaz.
- Validación con Zod para todo dato que entre por formulario.
- Arquitectura hexagonal: separar core de infraestructura.
- Repositorio con patrón adaptador (para poder cambiar el storage sin tocar la lógica).

Actividad / Proyecto a desarrollar.

Consigna grupal (2 estudiantes).

Deben desarrollar ArchiCode completo, aplicando todo lo visto. El sitio debe ser moderno, responsive y funcional 100% en frontend (sin backend, usamos localStorage como persistencia).

Entregables obligatorios.

1. Código fuente en repositorio GitHub (cada grupo).
2. Despliegue en Vercel (o Netlify/Railway) – la app debe estar online.
3. Informe técnico (PDF o Markdown) dentro del repo, que incluya:
 - Capturas de pantalla de la app funcionando.
 - Explicación de la arquitectura hexagonal usada.
 - Fragmentos de código destacados (Zod, generic, utility types).

Proyecto, Diseño e Implementación de Sitios Web Dinámicos.

- Dificultades encontradas y cómo las resolvieron.
- Link al proyecto desplegado.

4. Enlace de Live Share (VS Code) compartido con el profesor por comentario privado en la plataforma educativa antes de comenzar a codificar en clase.

5. Avance parcial subido a la plataforma (ZIP o Drive) al finalizar las 4 horas de clase, mostrando al menos 60% del proyecto funcionando. El 40% restante es polish, más validaciones, diseño y despliegue.

Criterios de evaluación.

Criterio	Peso	Descripción
Uso correcto de TypeScript	20%	Tipado fuerte, interfaces, utility types, generics. Sin `any`.
Validación con Zod	15%	Esquemas definidos y usados en cada entrada de datos.
Arquitectura hexagonal	20%	Separación clara core/infraestructura, puertos y adaptadores.
Funcionalidad completa	15%	CRUD de componentes, validación de patrones, persistencia.
Despliegue y GitHub	10%	Repo ordenado, commits descriptivos, Vercel funcionando.
Informe técnico	10%	Explicaciones claras, capturas, reflexión.
Live Share y avance en clase	10%	Conexión compartida a tiempo, avance del 60% al finalizar clase.

Nota: La falta de Live Share al inicio o avance menor al 60% en clase puede reducir hasta 2 puntos la nota final.

Documentación final (Informe técnico).

Deben entregar un archivo `INFORME.md` (o PDF) en la raíz del repo con esta estructura:

Informe Técnico - ArchiCode

Integrantes

- Nombre Apellido (rol: frontend/arquitectura)
- Nombre Apellido (rol: validaciones/integración)

Link a producción

<https://archicode-grupo-x.vercel.app>

Capturas de pantalla

(insertar imágenes)

Explicación de la arquitectura

Describir cómo separamos core de infraestructura, qué puertos definimos y qué adaptadores implementamos.

Código destacado

Mostrar:

- Una interfaz con utility types
- Un generic utilizado
- Un esquema de Zod
- El adaptador de localStorage

Dificultades y soluciones

...

Conclusión

Qué aprendieron y cómo aplicarían esto en un proyecto real.